

9 Practical: Malware Detection Using Artificial Intelligence

Step 1: Load all necessary libraries

We import all necessary Python libraries required for data handling, preprocessing, classification, and evaluation.

```
# ML for CBS practicals.
# Make a copy of this notebook and place it under your own
"MyDrive/Colab Notebooks" folder
# Please do not the run code here.

#importing all the necessary libraries
import numpy as np
import pandas as pd
from sklearn.feature_selection import SelectFromModel
import sklearn.ensemble as ske
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier
```

Step 2: Load the dataset

We mount Google Drive and load the dataset using the correct separator “sep=|”, ensuring the file is read correctly.

```
#printing the dataset
dataset #not suitable for
```



	Name	md5	Machine	SizeOfOptionalHeader	Characteristics	MajorLinkerVersion
0						
1						
2						
3						
4						
...						
10534						
10535						
10536						
10537						
10538						

10539 rows x 7 columns

```
dataset = pd.read_csv(path_to_dataset, sep = '|') #using the sep
parameter to make sure dataset ican be read easier
dataset #printing the dataset
```

	Name	md5	Machine	SizeOfOptionalHeader	Characteristics	MajorLinkerVersion	MinorLinkerVersion
0	Windows.Internal.Shell.Broker.dll	09e83f1d1c99ad33009dbe6fb129c2d9	34404	240	8226	14	12
1	hidserv.dll	3030f19c6a73367d6d5eedd157f5d01a	34404	240	8226	14	12
2	DmApiSetExtImplDesktop.dll	8271846f8f5dd1cfdcaa957b1b9da1a2	34404	240	8226	14	12
3	FSResizerSetup27.exe	5802b4215566bb9593a736e945a28e99	332	224	271	6	0
4	asc-setup.exe	8cb1fb45489d065720285deecbcbad98	332	224	33167	2	25
...	...	...	...	...	...	...	...
10534	VirusShare_dd619efac1cea076816815b2f6220a6d	dd619efac1cea076816815b2f6220a6d	332	224	271	6	0
10535	VirusShare_c93fb2f6ef3a92dc0a0d33fc3f8668a9	c93fb2f6ef3a92dc0a0d33fc3f8668a9	332	224	258	14	11
10536	VirusShare_622e962ebf6da7ac26aee84873682bce	622e962ebf6da7ac26aee84873682bce	332	224	8450	9	0
10537	VirusShare_70111012bb702b4cc13c52f7ceb3df91	70111012bb702b4cc13c52f7ceb3df91	332	224	271	6	0
10538	VirusShare_d7be59aca809b313c1f5c53bcbf38214	d7be59aca809b313c1f5c53bcbf38214	332	224	770	12	0

Step 3: Explore the dataset

We check the shape, summary statistics, and distribution of the target variable.

```
dataset.shape #prinitng the shape
```

(10539, 57)

dataset.info() #printing the shape of the dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10539 entries, 0 to 10538
Data columns (total 57 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Name                                     10539 non-null  object
1   md5                                     10539 non-null  object
2   Machine                               10539 non-null  int64
3   SizeOfOptionalHeader                  10539 non-null  int64
4   Characteristics                       10539 non-null  int64
5   MajorLinkerVersion                   10539 non-null  int64
6   MinorLinkerVersion                   10539 non-null  int64
7   SizeOfCode                           10539 non-null  int64
8   SizeOfInitializedData                 10539 non-null  int64
9   SizeOfUninitializedData               10539 non-null  int64
10  AddressOfEntryPoint                   10539 non-null  int64
11  BaseOfCode                           10539 non-null  int64
12  BaseOfData                           10539 non-null  int64
13  ImageBase                            10539 non-null  int64
14  SectionAlignment                     10539 non-null  int64
15  FileAlignment                        10539 non-null  int64
16  MajorOperatingSystemVersion          10539 non-null  int64
17  MinorOperatingSystemVersion          10539 non-null  int64
18  MajorImageVersion                    10539 non-null  int64
19  MinorImageVersion                    10539 non-null  int64
20  MajorSubsystemVersion                10539 non-null  int64
21  MinorSubsystemVersion                10539 non-null  int64
22  SizeOfImage                          10539 non-null  int64
23  SizeOfHeaders                        10539 non-null  int64
24  CheckSum                             10539 non-null  int64
```

dataset.describe() #getting nasic statistics using the .decribe function

```
Machine  SizeOfOptionalHeader  Characteristics  MajorLinkerVersion  MinorLinkerVersion  SizeOfCode  SizeOfInitializedData  SizeOfUninitializedData  AddressOf
count  10539.000000          10539.000000      10539.000000      10539.000000      10539.000000  1.053900e+04          1.053900e+04          1.053900e+04
mean   10897.261979           228.981118        6690.892115        11.002277          8.951513      1.318032e+06          1.048309e+06          2.818828e+05
std    15760.025794           7.525192         10279.733735        7.423987          11.159629      3.752865e+07          2.655705e+07          1.527909e+07
min     332.000000           224.000000         2.000000          0.000000          0.000000      0.000000e+00          0.000000e+00          0.000000e+00
25%     332.000000           224.000000        258.000000         6.000000          0.000000      4.300800e+04          3.993600e+04          0.000000e+00
50%     332.000000           224.000000        271.000000        14.000000         11.000000      1.658880e+05          1.628160e+05          0.000000e+00
75%    34404.000000          240.000000       8226.000000        14.000000        12.000000      4.003840e+05          8.668160e+05          0.000000e+00
max    34404.000000          328.000000      41358.000000       127.000000       165.000000      1.766614e+09          1.918989e+09          1.070112e+09

8 rows x 55 columns
```

dataset['legitimate'].value_counts() #getting the number of samples each attack label

	count
legitimate	
0	6999
1	3540

dtype: int64

Step 4: Preprocessing

We remove unnecessary columns like 'Name' and 'md5' that are not useful for training the model.

```
#it is necessary to drop certain features we will drop Name, md5 and legitimate
delete = ['Name', 'md5']

dataset = dataset.drop(delete, axis=1) #dropping the aforementioned columns using the drop function
```

```
[26] #verify whether the columns have been dropped using .columns function
dataset.columns
```

```
Index(['Machine', 'SizeOfOptionalHeader', 'Characteristics',
      'MajorLinkerVersion', 'MinorLinkerVersion', 'SizeOfCode',
      'SizeOfInitializedData', 'SizeOfUninitializedData',
      'AddressOfEntryPoint', 'BaseOfCode', 'BaseOfData', 'ImageBase',
      'SectionAlignment', 'FileAlignment', 'MajorOperatingSystemVersion',
      'MinorOperatingSystemVersion', 'MajorImageVersion', 'MinorImageVersion',
      'MajorSubsystemVersion', 'MinorSubsystemVersion', 'SizeOfImage',
      'SizeOfHeaders', 'Checksum', 'Subsystem', 'DllCharacteristics',
      'SizeOfStackReserve', 'SizeOfStackCommit', 'SizeOfHeapReserve',
      'SizeOfHeapCommit', 'LoaderFlags', 'NumberOfRvaAndSizes', 'SectionsNb',
      'SectionsMeanEntropy', 'SectionsMinEntropy', 'SectionsMaxEntropy',
      'SectionsMeanRawsize', 'SectionsMinRawsize', 'SectionMaxRawsize',
      'SectionsMeanVirtualsize', 'SectionsMinVirtualsize',
      'SectionMaxVirtualsize', 'ImportsNbDLL', 'ImportsNb',
      'ImportsNbOrdinal', 'ExportNb', 'ResourcesNb', 'ResourcesMeanEntropy',
      'ResourcesMinEntropy', 'ResourcesMaxEntropy', 'ResourcesMeanSize',
      'ResourcesMinSize', 'ResourcesMaxSize', 'LoadConfigurationSize',
      'VersionInformationSize', 'legitimate'],
      dtype='object')
```

```
[27] dataset.shape #printing the shape of the dataset using .shape
```

```
(10539, 55)
```

Step 5: Preprocessing

We split the dataset into features (X) and target labels (Y)

```
#We will separate out the features and labels from the dataset and
store them in X and Y
X = dataset.drop('legitimate', axis = 1) #saving dataset features in X
by dropping labels
```

```
Y = dataset['legitimate'] # saving the labels in Y
X.shape #printing the shape of X that stores the features using .shape
(10539, 54)
```

```
X #printing x
```

	machine	sizeofoptionalheader	characteristics	majorlinkerversion	minorlinkerversion	sizeofcode	sizeofinitializeddata	sizeofuninitializeddata	addressentr
0	34404	240	8226	14	12	779776	253952	0	
1	34404	240	8226	14	12	21504	13312	0	
2	34404	240	8226	14	12	33792	27648	0	
3	332	224	271	6	0	23552	164864	1024	
4	332	224	33167	2	25	87040	71680	0	
...	...	...	...	...	...	...	...	...	...
10534	332	224	271	6	0	40960	32768	0	
10535	332	224	258	14	11	286720	1044992	0	
10536	332	224	8450	9	0	89600	55808	0	
10537	332	224	271	6	0	24576	141824	2048	
10538	332	224	770	12	0	28672	4096	65536	

10539 rows x 54 columns

```
Y.shape #printing Y, that stores labels using .shape
(10539,)
```

```
Y #printing the labels
```

	legitimate
0	1
1	1
2	1
3	1
4	1
...	...
10534	0
10535	0
10536	0
10537	0
10538	0

10539 rows x 1 columns

dtype: int64

Step 6: Preprocessing

We use ExtraTreesClassifier to select the most important features for training.

```
▶ X_new #printing the transformed version of x
```

```
array([[3.44040000e+04, 2.40000000e+02, 8.22600000e+03, ...,  
       3.43726791e+00, 2.56000000e+02, 1.60000000e+01],  
       [3.44040000e+04, 2.40000000e+02, 8.22600000e+03, ...,  
       3.46549876e+00, 2.56000000e+02, 1.60000000e+01],  
       [3.44040000e+04, 2.40000000e+02, 8.22600000e+03, ...,  
       3.46647369e+00, 2.56000000e+02, 1.60000000e+01],  
       ...,  
       [3.32000000e+02, 2.24000000e+02, 8.45000000e+03, ...,  
       5.02069508e+00, 7.20000000e+01, 1.60000000e+01],  
       [3.32000000e+02, 2.24000000e+02, 2.71000000e+02, ...,  
       5.26917350e+00, 0.00000000e+00, 0.00000000e+00],  
       [3.32000000e+02, 2.24000000e+02, 7.70000000e+02, ...,  
       4.91161452e+00, 0.00000000e+00, 0.00000000e+00]])
```

```
[39] X_new.shape #printing the shape of X_new using .shape
```

```
▶ (10539, 10)
```

```
[40] #store the number of features using number_of_features  
number_of_features = X_new.shape[1]
```

```
[41] #printitng number of features  
number_of_features
```

```
▶ 10
```

```
▶ #we will use tree based feature selection  
feature_selection = ske.ExtraTreesClassifier().fit(X,Y)
```

```
[34] #printing feature selection (Extra Tree Classifier)  
feature_selection
```

```
▶ ExtraTreesClassifier  
ExtraTreesClassifier()
```

```
[35] # Store select from model in a variable called model  
model = SelectFromModel(feature_selection, prefit = True)
```

```
[36] model #printing the model
```

```
▶ SelectFromModel  
  estimator:  
    ExtraTreesClassifier  
      ▶ ExtraTreesClassifier
```

```
[37] #new set of features are stored in X_new  
X_new = model.transform(X)
```

```
▶ /usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but SelectFromModel was fitted without feature names  
warnings.warn()
```

```
14] feature_selection.feature_importances_[:-1] #printing the same features
```

```
array([3.27516496e-02, 3.22452377e-02, 3.17602918e-03, 8.26505652e-03,
       3.50591247e-03, 2.24841987e-02, 1.71939302e-02, 1.14097999e-02,
       4.28387136e-03, 6.46404132e-04, 2.00945743e-03, 4.68606555e-03,
       1.99503439e-02, 2.83300565e-03, 2.61416378e-03, 3.10708173e-03,
       3.50134009e-03, 2.88848366e-03, 3.85873108e-03, 1.39552986e-02,
       5.64242584e-03, 7.49215853e-03, 3.97169327e-03, 4.22425595e-05,
       2.36430347e-07, 1.52417659e-04, 1.16488224e-03, 2.31565959e-03,
       2.41640116e-03, 3.63439509e-02, 1.36468229e-02, 5.11615574e-03,
       3.94908377e-03, 2.64631025e-03, 2.17298775e-03, 6.07230442e-02,
       1.96776185e-03, 7.20262213e-03, 9.00775177e-03, 1.09275230e-01,
       6.04633967e-03, 2.03083474e-03, 1.51929249e-02, 3.04518425e-03,
       7.71502635e-04, 5.76925173e-03, 1.35910757e-03, 2.84376714e-03,
       5.69109808e-03, 6.32338894e-03, 6.88424836e-03, 8.27202756e-02,
       6.87134175e-02, 3.21992759e-01])
```

Step 7: Preprocessing

We split the selected features into training and testing sets (80/20 ratio).

```
[45] # Splitting the dataset into the Training set and Test set
X_train, X_test, y_train, Y_test = train_test_split(X_new, Y, test_size = 0.20, random_state = 0)
```

```
[46] X_train.shape #printing shape of x_train using .shape
```

```
(8431, 10)
```

```
X_train #printing x_train
```

```
array([[3.32000000e+02, 2.24000000e+02, 2.58000000e+02, ...,
       7.99964305e+00, 9.20000000e+01, 1.60000000e+01],
       [3.32000000e+02, 2.24000000e+02, 3.31670000e+04, ...,
       7.98420549e+00, 0.00000000e+00, 0.00000000e+00],
       [3.32000000e+02, 2.24000000e+02, 3.31670000e+04, ...,
       5.13965300e+00, 0.00000000e+00, 1.50000000e+01],
       ...,
       [3.44040000e+04, 2.40000000e+02, 8.22600000e+03, ...,
       3.47732315e+00, 2.56000000e+02, 1.60000000e+01],
       [3.32000000e+02, 2.24000000e+02, 2.70000000e+02, ...,
       6.87835011e+00, 0.00000000e+00, 1.80000000e+01],
       [3.44040000e+04, 2.40000000e+02, 8.22600000e+03, ...,
       3.62099111e+00, 2.56000000e+02, 1.60000000e+01]])
```

```
[48] y_train.shape #printing y.shape using .shape (stores all features)
```

```
(8431,)
```

```
y_train #print the labels in y_train
```

legitimate	
4974	0
5919	0
5014	0
5186	0
1282	1
...	...
9225	0
4859	0
3264	1
9845	0
2732	1

8431 rows x 1 columns

dtype: int64

```
[50] X_test.shape #printing x_test shape using .shape
```

```
(2108, 10)
```

```
Y_test.shape #printing the shape of y_test
```

```
(2108,)
```

```
Y_test #printing y_test
```

legitimate	
6049	0
10262	0
3345	1
2578	1
8992	0
...	...
1469	1
6275	0
1944	1
7508	0
143	1

2108 rows x 1 columns

dtype: int64

Step 8: Preprocessing

We normalize the features using StandardScaler so that all features have similar scales.

Step 8: Preprocessing

```
0 dek # Feature StandardScaler: sc
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train) #scaling the features in X_train
X_test = sc.transform(X_test) #scaling the features in X_test
```

```
0 dek [68] X_train.shape #printing the shape of X_train using .shape
```

```
(8431, 10)
```

```
0 dek [56] X_train #printing X_train
```

```
array([[ -0.67295557, -0.6624437 , -0.62444437, ...,  1.32321059,
        -0.01089221,  0.67342775],
       [ -0.67295557, -0.6624437 ,  2.60873372, ...,  1.3150595 ,
        -0.01089749, -1.52817929],
       [ -0.67295557, -0.6624437 ,  2.60873372, ..., -0.18687624,
        -0.01089749,  0.53582731],
       ...,
       [ 1.48598219,  1.45230937,  0.15837987, ..., -1.06459343,
        -0.01088279,  0.67342775],
       [ -0.67295557, -0.6624437 , -0.62326542, ...,  0.73116318,
        -0.01089749,  0.94862863],
       [ 1.48598219,  1.45230937,  0.15837987, ..., -0.98873614,
        -0.01088279,  0.67342775]])
```

```
0 dek [57] X_test.shape #printing the shape of X_test using .shape
```

```
(2108, 10)
```

```
[58] X_test #printing X_test
```

```
array([[ -0.67295557, -0.6624437 , -0.62444437, ..., -0.38295836,
        -0.01089221, -1.52817929],
       [ -0.67295557, -0.6624437 , -0.62316718, ..., -0.01513241,
        -0.01089749,  0.26062643],
       [ 1.48598219,  1.45230937,  0.15837987, ..., -1.065661 ,
        -0.01088279,  0.67342775],
       ...,
       [ 1.48598219,  1.45230937,  0.15837987, ..., -1.07657045,
        -0.01088279,  0.67342775],
       [ -0.67295557, -0.6624437 , -0.62444437, ..., -0.26002394,
        -0.01089749,  0.67342775],
       [ -0.67295557, -0.6624437 ,  2.60873372, ...,  1.31604286,
        -0.01089749,  0.81102819]])
```

Step 9: Train classifier models

We train a Random Forest Classifier using the training data and evaluate it on the test set.

```

from sklearn.ensemble import RandomForestClassifier #importing the
random tree classifier
classifier = RandomForestClassifier(n_estimators = 50, criterion =
'entropy') #setting up 50 trees and using ectropy as criterion
classifier.fit(X_train, y_train) #train model using X_train and y_train

print("Model has been trained") #statment that will print once training
has completed

```

➤ Model has been trained

▼ Predict and Evaluate - Random Forest

```

✓ [60] #predict the test results
0 cek. y_pred = classifier.predict(X_test)

```

```

✓ [61] #printing y_pred
0 cek. y_pred
array([0, 0, 1, ..., 1, 0, 0])

```

```

✓ [62] #Makeing the confusion matrix
0 cek. from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, y_pred)
print(cm)
print(classification_report(Y_test, y_pred))

```

```

[[1400  17]
 [ 19 672]]

```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1417
1	0.98	0.97	0.97	691
accuracy			0.98	2108
macro avg	0.98	0.98	0.98	2108
weighted avg	0.98	0.98	0.98	2108

Step 10: Visualize One Tree from the Random Forest

We visualize one decision tree from the trained Random Forest model using Graphviz.

```

# Step 10: Visualize one tree from the trained Random Forest

from sklearn import tree
import graphviz

# Extracting the first decision tree from the Random Forest

```

```

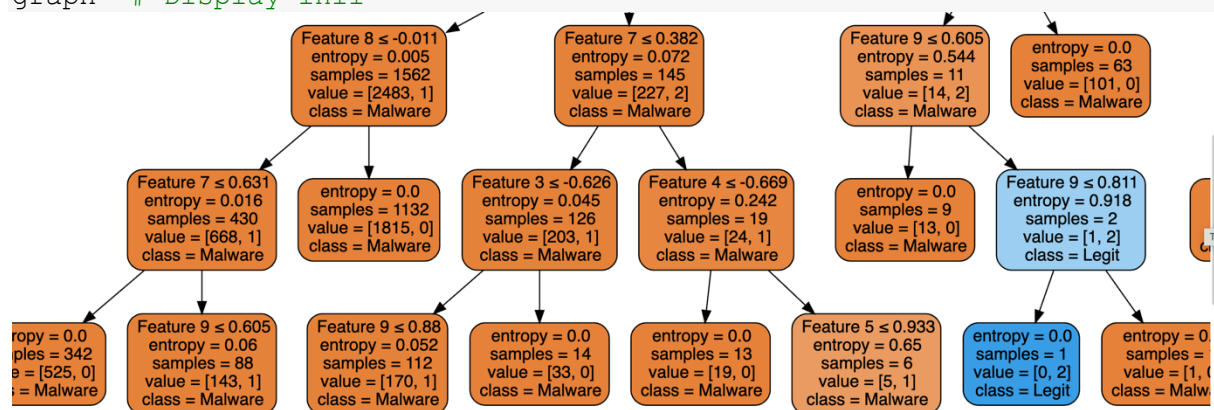
estimator = classifier.estimators_[0]

# If feature names are not available (after feature selection),
generate dummy names
feature_names = [f"Feature {i}" for i in range(X_train.shape[1])]

# Export the tree to DOT format
dot_data = tree.export_graphviz(estimator,
                                out_file=None,
                                feature_names=feature_names,
                                class_names=["Malware", "Legit"],
                                filled=True,
                                rounded=True,
                                special_characters=True)

# Create and render the graph
graph = graphviz.Source(dot_data)
graph.render("random_forest_tree") # Save as PNG file
graph # Display inli

```



Step 11: Train and Evaluate K-Nearest Neighbors (KNN)

We train a K-Nearest Neighbors model and evaluate its performance using the same test set.

```

from sklearn.neighbors import KNeighborsClassifier
classifier_KNN = KNeighborsClassifier(n_neighbors = 3, metric =
'minkowski', p = 2)
classifier_KNN.fit(X_train, y_train)

print("KNN Model has been trained")

```

✓ KNN Model has been trained

✓ Predict and Evaluate - KNN

```
✓ [64] #predict the test results
)
!K. y_pred_KNN = classifier_KNN.predict(X_test)
```

```
✓ [65] y_pred_KNN
)
!K. ➡ array([0, 0, 1, ..., 1, 0, 0])
```

```
✓ [66] #Makeing the confusion matrix
)
!K. from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, y_pred_KNN)
print(cm)
print(classification_report(Y_test, y_pred_KNN))
```

```
➡ [[1390  27]
   [ 27 664]]
```

		precision	recall	f1-score	support
	0	0.98	0.98	0.98	1417
	1	0.96	0.96	0.96	691
	accuracy			0.97	2108
	macro avg	0.97	0.97	0.97	2108
	weighted avg	0.97	0.97	0.97	2108

Conclusion:

In this practical, we successfully trained and evaluated two machine learning models — Random Forest and K-Nearest Neighbors (KNN) — for malware detection. The Random Forest model showed slightly better performance in terms of precision, recall, and accuracy. We also visualized one decision tree from the forest to better understand how the model works. Overall, both models were effective, but Random Forest was more accurate and reliable for this task.